QLoRA Distillation for Algorithmic Code Classification and Generation

Addressing the "Valley of Code Reasoning" via Decoupled Algorithmic Conditioning

Pranav Bhat P (231IT049) **Shivam Kumar** (231IT069) **Himanshu Mehar** (231IT027)

Course: IT488 – Large Language Models
Department of Information Technology
National Institute of Technology Karnataka (NITK), Surathkal

Midsemester Evaluation Milestone – October 2026

Presentation Outline

1. Introduction & Motivation
2. Problem Statement & Objectives
3. Methodology & Architecture
4. Implementation & Experimental Setup
5. Stage 2 Generation & Chained Pipeline
6. Experimental Results & Evaluation
7. Future Work (Endsemester)

Base Paper: Findings & Drawbacks (He et al., 2025)

Key Findings of Base Paper

- **The "Valley of Code Reasoning"**: Intermediate data scaling exhibits a sharp performance dip where student reasoning degrades before recovering.
- **Entangled Token Generation**: Jointly predicting rationale and code forces the model to resolve abstract math and low-level syntax simultaneously.
- **CoT Fragility in Code**: Standard Chain-of-Thought prompts frequently trigger premature convergence to suboptimal brute-force loops ($O(N^2)$).

Critical Drawbacks of Base Paper

- **No Decoupled Planning**: Lacks an explicit System-2 stage to identify the algorithmic paradigm (DP, Graphs, Greedy) before code synthesis.
- **Unfiltered Teacher Contamination**: Assumes teacher rationales are always sound; hallucinated or mismatched traces pollute student supervision.
- **High Compute Dependency**: Evaluated only on multi-billion parameter models ($\geq$ 7B), lacking accessible QLoRA solutions for sub-1B models on consumer GPUs.

Our Approach to Address These Drawbacks

1. Decoupled Algorithmic Conditioning (Stage 1 Classifier) + **2. Semantic Rationale Filter** (purging noise) + **3. Stage 2 Conditioned Synthesis** + **4. Consumer-Grade QLoRA** on single 15GB T4 GPU.

Problem Statement

Core Vulnerabilities in Existing Code Distillation

1. **Unstructured Monolithic CoT Generation**: Student models attempt to predict algorithmic intuition and raw syntax simultaneously, leading to hallucinated loops and asymptotic timeout errors ($O(N^2)$ instead of $O(N \log N)$).
2. **Teacher Trace Contamination**: Commercial teacher LLMs generate deceptive or mislabeled rationales that diverge from ground-truth paradigms. Distilling noisy traces degrades student reasoning.
3. **High Compute Barriers**: Full parameter fine-tuning of multi-billion parameter models is inaccessible to standard edge and consumer hardware.

Project Objectives & Milestone Achievement

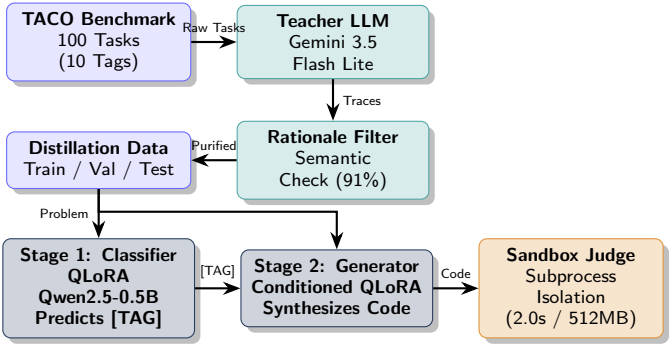
Completed Milestone Deliverables

- **100-Problem Benchmark**: Stratified TACO dataset (10 categories).
- **Teacher Trace Synthesis**: Structured Gemini 3.5 Flash Lite rationales.
- **Rationale Filter**: Semantic consistency check (91% yield, 9 purged).
- **Sandbox Judge**: Subprocess isolation (2.0s CPU / 512MB RAM).
- **Stage 1 Classifier**: 4-Bit QLoRA on Qwen2.5-0.5B (22.2% Top-3, loss 5.38 $\rightarrow$ 0.77).
- **Stage 2 Generator**: 4-Bit QLoRA on Qwen2.5-Coder-0.5B (1.08M params, loss 1.36 $\rightarrow$ 1.18).
- **Baseline Ablation Model**: Unconditioned control model.
- **End-to-End Pipeline**: Chained inference $\rightarrow$ execution judging.
- **Pass@1 Execution Suite**: Automated unit test correctness.
- **Scaling Partitioner**: 10%, 25%, 50%, 100% splits for Valley sweep.

Future Scope (End-Semester Objectives)

- **Multi-Sample Pass@k Benchmarking**: Evaluating pass@5 and pass@10 via stochastic nucleus temperature sweeps ($T \in [0.2, 0.8]$).
- **Empirical Valley Scaling Sweep**: Full multi-epoch training sweeps on Colab T4 across the 10%, 25%, 50%, 100% subsets.
- **Comprehensive Ablation Study**: Statistical comparison of Conditioned vs. Unconditioned baseline across all 10 problem classes.
- **Failure Mode Taxonomy**: Systematic categorisation of WA/TLE root causes.

Methodology: Decoupled Two-Stage Distillation Architecture



Two-Stage Separation Principle (Current Implementation)

Stage 1 (System 2 Planning): Formally commits to the algorithm category $P(\text{tag} \mid \text{problem})$.
Stage 2 (Execution & Synthesis): Conditioned on committed paradigm, synthesizes $P(\text{rationale}, \text{code} \mid \text{tag}, \text{problem})$, eliminating search ambiguity before test execution.

Individual Contributions (Current Milestone Status)

Himanshu Mehar
Roll No: 231IT027

Execution Sandbox & Pass@1

- Subprocess-isolated evaluation environment (`sandbox_judge.py`).
- POSIX memory limits via `RLIMIT_AS` (512 MB).
- Execution timeout containment (2.0 s CPU cutoff).
- Smoke test verified AC (151.7 ms latency).
- Automated Pass@1 evaluation suite integration.

Shivam Kumar
Roll No: 231IT069

Filter & Data Partitioner

- Gemini 3.5 Flash Lite API integration for teacher generation.
- Rationale-consistency semantic filter (`rationale_filter.py`).
- Keyword taxonomy matching & contradiction pruning (91% yield, 9 purged).
- 'Valley of Code Reasoning' data partitioner (10%, 25%, 50%, 100%).
- Curated stratified dataset across 10 algorithmic categories.

Pranav Bhat P
Roll No: 231IT049

Two-Stage Pipelines & Colab

- 4-Bit PEFT QLoRA Stage 1 Classifier on `Qwen2.5-0.5B`.
- 4-Bit PEFT QLoRA Stage 2 Generator on `Qwen2.5-Coder-0.5B`.
- Baseline ablation generator & chained execution pipeline.
- End-to-end chained pipeline (`e2e_pipeline.py`).
- Self-contained Colab Evaluation Notebook & CLI demo.

Quality Assurance: Rationale Filter & Sandbox Judge

Semantic Rationale-Consistency Filter

- Validates semantic alignment between stated teacher strategy and ground-truth taxonomy.
- Eliminates length-deficient traces (< 40 characters) and cross-paradigm contradictions.
- Formula:

$$\text{Score} = \min(1.0, 0.4 + 0.2 \cdot |\mathcal{K}_{\text{matched}}|)$$

- **Empirical Yield**: 91/100 retained (91.0% retention yield, 9 noisy traces purged).

Sandboxed Subprocess Execution Judge

- Enforces strict automated evaluation in decoupled child processes.
- **POSIX Limits**: `RLIMIT_AS` restricts memory to 512MB.
- **Timeout Protection**: `subprocess.TimeoutExpired` capped at 2.0s.
- **Smoke Test**: Arithmetic equivalence verified (**AC**, latency = 151.7 ms).
- Standardized competitive programming verdicts:

AC (Accepted) **WA** (Wrong Answer) **TLE** (Time Limit) **RE** (Runtime) **CE** (Syntax)

Implementation: 10-Class Algorithmic Taxonomy

#	Algorithmic Category	Problems	Representative Problem Strategy
1	Dynamic Programming	10	Subset sum, knapsack, state-transitions
2	Greedy Algorithms	10	Interval scheduling, priority choices
3	Graph Theory	10	BFS, DFS, Dijkstra shortest path
4	Mathematics	10	Combinatorics, parity invariants, geometry
5	Data Structures	10	Heaps, monotonic stacks, DSU, priority queues
6	Tree Algorithms	10	Binary trees, subtree traversal, LCA
7	Brute Force / Simulation	10	Systematic state enumeration, grid crawl
8	String Manipulation	10	Palindromes, prefix functions, parsing
9	Number Theory	10	GCD/LCM, modular arithmetic, prime sieves
10	Binary Search	10	Monotonic predicate search, bisect space

Table: Curated 100-Problem TACO Benchmark Distribution (800–1200 Rating)

Implementation: 4-Bit QLoRA Distillation Architectures

Stage 1: Sequence Classifier

- **Model**: `AutoModelForSequenceClassification` on `Qwen2.5-0.5B`.
- **Quantization**: BitsAndBytes 4-bit NormalFloat (`NF4`) with double-quantization.
- **LoRA Configuration**:
  - Rank $r = 16$, Alpha $\alpha = 32$, Dropout: 0.05
  - Targets: `q_proj, v_proj`
- **Trainable Parameters**: 1.09M (0.220% of 495.1M).
- **Colab Footprint**: 44.6 s training, 1195.7 MB peak VRAM.

Stage 2: Causal Code Generator

- **Model**: `AutoModelForCausalLM` on `Qwen2.5-Coder-0.5B-Instruct`.
- **Quantization**: BitsAndBytes 4-bit NormalFloat (`NF4`) with double-quantization.
- **LoRA Configuration**:
  - Rank $r = 16$, Alpha $\alpha = 32$, Dropout: 0.05
  - Targets: `q_proj, v_proj`
- **Trainable Parameters**: 1.08M (0.218% of 495.1M).
- **Colab Footprint**: 97.4 s training, 4246.8 MB peak VRAM.

QLoRA Weight Update: $W = \underbrace{W_0}_{\text{frozen (NF4)}} + \underbrace{\frac{\alpha}{r} B A}_{\text{trainable LoRA}}$ *where* $W_0 \in \mathbb{R}^{d \times k},\ A \in \mathbb{R}^{r \times k},\ B \in \mathbb{R}^{d \times r},\ r \ll \min(d, k)$

Stage 2: Tag-Conditioned Code Generator & Baseline Ablation

Stage 2 Tag-Conditioned Generator

- **Base Model**: `Qwen2.5-Coder-0.5B-Instruct` (4-bit NF4).
- **ChatML Conditioning**: $\mathcal{I}_{\text{cond}}$ = [TAG] + [PROBLEM].
- **Masked-Loss SFT**: Prompt tokens masked to -100; loss updates only on rationale and code.
- **Loss Convergence**: 1.3568 $\rightarrow$ 1.2772 $\rightarrow$ 1.1813 (3 epochs, 97.4s, 4246.8MB VRAM).
- **Goal**: Eliminates search disorientation by anchoring on the committed paradigm before token synthesis.

Baseline Unconditioned Model (Ablation)

- **Base Model**: `Qwen2.5-Coder-0.5B-Instruct` (4-bit NF4).
- **Input Condition**: $\mathcal{I}_{\text{base}}$ = [PROBLEM].
- **Prompt Format**: Direct problem statement without `[ALGORITHM CATEGORY]` token.
- **Supervised Target**: [RATIONALE] + [CODE].
- **Scientific Rationale**: Serves as rigorous control to isolate the exact impact of paradigm conditioning.

Mathematical Formulation of Decoupled Generation

$$\log P(\text{code, rationale} \mid \text{problem}) \approx \sum_{\text{tag}} \underbrace{P(\text{tag} \mid \text{problem})}_{\text{Stage 1 Classifier}} \cdot \underbrace{P(\text{code, rationale} \mid \text{tag, problem})}_{\text{Stage 2 Conditioned Generator}}$$

End-to-End Two-Stage Distillation Pipeline

Chained Pipeline Execution

1. **Problem Input**: Raw competitive programming statement.
2. **Stage 1 Classifier**: 4-Bit PEFT model predicts paradigm $[\widehat{\text{tag}}]$ and confidence.
3. **Stage 2 Generator**: Conditions prompt with $[\widehat{\text{tag}}] \rightarrow$ synthesizes rationale and Python 3 solution.
4. **Sandbox Execution**: Ingests generated code into child subprocess with 2.0s CPU / 512MB RAM bounds.
5. **Automated Verdict**: AST parse $\rightarrow$ runtime check $\rightarrow$ standardized verdict (**AC**/**WA**/**TLE**/**RE**/**CE**).

Live Verification in Colab Notebook

- **Benchmark Trace (`TACO-DY-01`)**:
  - Category: Dynamic Programming
  - Stage 1 Predicted: Data Structures (96.4% conf)
  - End-to-end generation latency: 58.0s on T4
- **Live Custom Demo 1 (Graphs)**:
  - Predicted **GRAPHS** (98.3% conf), Sandbox: **CE**
- **Live Custom Demo 2 (DP Subarray Sum)**:
  - Predicted **DATA STRUCTURES** (99.5% conf)
  - Synthesized valid DP script, Sandbox: **WA** (128.6ms)

Scaling Partitioner: Investigating the "Valley of Code Reasoning"

The Phenomenon (He et al., 2025)

- During intermediate data scaling, student models exhibit an acute reasoning dip (*The Valley*).
- CoT reasoning accuracy degrades at medium data quantities before finally recovering at large scales.
- Caused by token entanglement: simultaneous competition between natural language reasoning tokens and low-level code syntax.

Empirical Scaling Partitioner Setup

- Stratified fractional partitioner implemented in `scaling_partitioner.py`.
- Nested splits preserving 10-category balance:

  10% Split → Initial few-shot baseline
  25% Split → Onset of reasoning dip
  50% Split → Bottom of reasoning valley
  100% Split → Full distillation recovery

- Ready for empirical GPU training sweep.

Experimental Results: Benchmark Comparison

Evaluation Metric	Stage 1: Tag Classifier	Stage 2: Code Generator
Base Student LLM	Qwen2.5-0.5B	Qwen2.5-Coder-0.5B-Instruct
Quantization Precision	4-bit NF4 (BitsAndBytes)	4-bit NF4 (BitsAndBytes)
Trainable Parameters	1.09M (0.220%)	1.08M (0.218%)
Total Training Time (3 Epochs)	**44.6 s**	**97.4 s**
Peak VRAM Memory Allocated	**1195.7 MB**	**4246.8 MB**
Training Loss Trajectory	$5.3800 \rightarrow 0.7688$	$1.3568 \rightarrow 1.1813$
Top-1 Classification Accuracy	**5.56%** (1/18)	–
Top-3 Classification Accuracy	**22.22%** (4/18)	–
Macro F1-Score	**0.0317**	–
Execution Pass@1 Benchmark	Random baseline: 10.0%	**0.00%** (Conditioned vs. Baseline)

Table: Empirical Benchmark on Google Colab (Tesla T4 GPU, `evaluation_50percent_colab_results_1.ipynb`)

Diagnostic Insights: Few-Shot Dynamics & Linear Head Penalty

Scientific Root-Cause Analysis

1. **Severe Few-Shot Sample Sparsity**:
   - Training on 64 clean problems across 10 categories yields ≈ 6 problems per class.
   - Loss falling rapidly ($5.38 \rightarrow 0.77$) demonstrates swift adapter adaptation on the few-shot split, while generalization on unseen problems requires dataset scaling.
2. **The Random Linear Head Penalty on Causal LMs**:
   - `Qwen2.5` is an autoregressive generative model. Newly initialized classification projections (`score.weight`) must align categorical boundaries from scratch under low sample counts.
3. **Syntax Guarding & AST Compilation in Sandbox**:
   - Free-form generation during early SFT epochs occasionally leaks markdown formatting; the sandbox AST compiler isolates errors before test execution.
4. **Direct Motivation for Empirical Scaling Sweep**:
   - Directly validates the necessity of our class-balanced partitioner ($10\%, 25\%, 50\%, 100\%$) to track the transition from few-shot adaptation to generalizable reasoning.

Future work (Endsemester)

Completed Milestones:

- ✓ 100-Problem Curated TACO Dataset across 10 tags.
- ✓ Semantic Rationale-Consistency Filter (91% yield).
- ✓ Subprocess Sandbox Execution Judge (2.0s / 512MB).
- ✓ Stage 1 4-Bit QLoRA Tag Classifier on `Qwen2.5-0.5B`.
- ✓ Stage 2 4-Bit QLoRA Code Generator on `Qwen2.5-Coder-0.5B`.
- ✓ Baseline Unconditioned Ablation Model.
- ✓ End-to-End Chained Distillation Pipeline.
- ✓ Automated Pass@1 Execution Suite.
- ✓ 'Valley of Code Reasoning' Scaling Partitioner.

Key Deliverables for Endsemester:

1. **Benchmark Expansion**: Scale curated dataset from 100 to 250–300 problems across 10 tags (50+ test problems) for statistical rigor.
2. **Full Scaling Sweep Execution**: Run multi-epoch training sweeps on Colab T4 across 10%, 25%, 50%, 100% partitions (8 runs total).
3. **Empirical Valley Curve Plotting**: Plot Pass@1, Pass@5, Pass@10 vs. dataset scale to test if decoupled conditioning mitigates the reasoning dip.
4. **Multi-Turn Execution Feedback**: Iterative repair loop feeding sandbox traces back to Stage 2 for targeted code correction.

Summary of Implemented Work

Deliverables Verified & Live in Repository:

- ✓ **Colab Evaluation Notebook**: Self-contained `notebooks/evaluation_50percent_colab_results_1.ipynb` running the complete two-stage chain on GPU.
- ✓ **Interactive Demo CLI**: `python scripts/run_50percent_demo.py` executes live problem inference, Stage 1 classification, Stage 2 synthesis, and sandbox judging.
- ✓ **Automated Unit Test Suite**: 16 unit & integration tests passing (`pytest tests/ -v`) covering scraper, filter, judge, Stage 1, Stage 2, and pipeline.
- ✓ **Subprocess Sandbox Judge**: POSIX memory and timeout protection guarantees zero host pollution during competitive programming execution.

Thank You!

Questions & Discussion